Pandas 数据处理实战手册

从零基础到竞赛实战

竞赛培训组

2026 年 3 月 29 日

学习指引

后续课程：学完 Pandas 后，下一步是部署大数据平台。

配套练习：练习数据已包含在本手册的 CSV 文件中。更多练习见仓库根目录的《仿真实战练习.md》。

目录

1 前言：为什么要学习 Pandas？

什么是 Pandas？

Pandas 是 Python 语言中最流行的数据处理和分析库，它的名字来源于"Panel Data"（面板数据）的缩写。Pandas 提供了高效、灵活的数据结构，特别适合处理结构化数据（如表格数据）。

核心特点：

- **DataFrame**：二维表格数据结构，类似 Excel 表格
- **Series**：一维数组数据结构，类似表格中的一列
- 强大的数据读取、清洗、转换、分析功能
- 与 NumPy、Matplotlib 等库无缝集成

为什么数据处理很重要？

在真实的数据分析项目中，数据科学家通常会将 **70%-80%** 的时间花在数据清洗和预处理上，只有 20%-30% 的时间用于建模和分析。

常见数据质量问题：

1. **缺失值**：某些记录缺少某些字段的值
2. **重复值**：同一条数据被记录了多次
3. **异常值**：明显不合理的数据（如年龄为负数）
4. **格式不一致**：同一类数据有多种表示方式
5. **数据类型错误**：数字被存储为字符串

竞赛评分标准：在大数据竞赛中，数据预处理通常占 **30%** 的评分权重，是获得高分的关键！

2 四个练习数据文件介绍

2.1 数据文件概览

本手册所有示例代码都围绕以下**四个 CSV 文件**展开，请确保这些文件和代码在同一目录下：

表 1: 练习数据文件说明

文件名	数据量	主题	主要问题
01_ 电商订单数据 _ 练习.csv	50 行	电商订单	重复、缺失、异常值、格式混乱
02_ 传感器数据 _ 练习.csv	144 行	传感器读数	时间序列、格式问题
03_ 缺失值处理 _ 练习.csv	30 行	员工信息	各种缺失值场景
04_ 数据格式清洗 _ 练习.csv	30 行	用户信息	格式不统一、单位混乱

2.2 01_ 电商订单数据 _ 练习.csv

数据字段说明

- **order_id**：订单编号
- **user_id**：用户 ID（部分缺失）
- **product_name**：商品名称
- **price**：价格（带”元”单位）
- **quantity**：数量（有负数异常值）
- **order_date**：订单日期（多种格式）
- **address**：地址
- **phone**：手机号（带横线）
- **tags**：商品标签（JSON 格式）

2.3 02_ 传感器数据 _ 练习.csv

数据字段说明

- **timestamp**：时间戳
- **sensor_id**：传感器 ID
- **temperature**：温度
- **humidity**：湿度（部分缺失）
- **pressure**：压力
- **status**：设备状态

2.4 03_ 缺失值处理 _ 练习.csv

数据字段说明

- **id**：编号
- **name**：姓名
- **age**：年龄（部分缺失）
- **gender**：性别（部分缺失）
- **salary**：工资（部分缺失）
- **department**：部门（部分缺失）
- **join_date**：入职日期（部分缺失）
- **phone**：电话
- **email**：邮箱
- **score**：评分（部分缺失）

2.5 04_ 数据格式清洗 _ 练习.csv

数据字段说明

- **id**：编号
- **name**：姓名（带空格）
- **birth_date**：出生日期（多种格式）
- **height**：身高（带单位）
- **weight**：体重（带单位）
- **income**：收入（带 k 单位）
- **phone**：电话
- **address**：地址
- **product_tags**：商品标签

3 Python 与 Pandas 基础

3.1 安装与导入

什么是 Python 包？

Python 的强大之处在于它有丰富的第三方库（包）。这些库是由全球开发者贡献的，可以帮助我们快速实现各种功能，而不需要从零开始编写代码。

常用数据科学包：

- **pandas**：数据处理和分析
- **numpy**：数值计算
- **matplotlib**：数据可视化
- **scikit-learn**：机器学习

```
# 在命令行中运行
pip install pandas
```

Listing 1: 安装 Pandas

```
import pandas as pd
import numpy as np
```

Listing 2: 导入 Pandas

导入别名约定

在 Python 数据科学社区中，有一些约定俗成的导入别名：

- `import pandas as pd` ——pd 代表 pandas
- `import numpy as np` ——np 代表 numpy

使用别名可以让代码更简洁，这也是行业内的标准做法。

3.2 读取 CSV 文件

什么是 CSV 文件？

CSV（Comma-Separated Values，逗号分隔值）是一种通用的、简单的数据存储格式。

CSV 的特点：

- 纯文本格式，可以用任何文本编辑器打开
- 每行是一条记录，字段之间用逗号分隔
- 第一行通常是列名（表头）
- 可以被 Excel 直接打开和编辑
- 文件体积小，兼容性好

示例 CSV 内容：

```
name,age,city
张三,25,北京
李四,30,上海
王五,28,广州
```

知识点：pd.read_csv()

`pd.read_csv()` 是 Pandas 中最常用的函数，用于读取 CSV 文件并转换为 DataFrame 对象。

基本语法：

```
df = pd.read_csv('文件路径.csv')
```

常用参数：

- `encoding`：指定文件编码，如'utf-8'、'gbk'
- `header`：指定哪一行作为列名，默认为 0（第一行）
- `sep`：指定分隔符，默认为逗号

```python
# 读取电商订单数据
df_order = pd.read_csv('01_电商订单数据_练习.csv')

# 读取传感器数据
```

```python
df_sensor = pd.read_csv('02_传感器数据_练习.csv')

# 读取缺失值练习数据
df_missing = pd.read_csv('03_缺失值处理_练习.csv')

# 读取格式清洗练习数据
df_messy = pd.read_csv('04_数据格式清洗_练习.csv')
```

Listing 3: 读取四个练习文件

动手练习 1：读取并查看数据

任务： 读取四个 CSV 文件，查看每个文件的基本信息。

```python
import pandas as pd

# 读取四个文件
df_order = pd.read_csv('01_电商订单数据_练习.csv')
df_sensor = pd.read_csv('02_传感器数据_练习.csv')
df_missing = pd.read_csv('03_缺失值处理_练习.csv')
df_messy = pd.read_csv('04_数据格式清洗_练习.csv')

# 查看电商订单数据的形状
print("电商订单数据形状：", df_order.shape)
print("\n前5行：")
print(df_order.head())
```

4 数据查看与探索

4.1 查看数据基本信息

为什么需要先查看数据？

在进行任何数据处理之前，我们必须先**了解数据**。就像医生看病需要先诊断一样，数据分析也需要先”诊断”数据的质量和结构。

数据探索的目的：

1. 了解数据规模（多少行、多少列）
2. 了解每列的数据类型

3. 发现明显的数据质量问题

4. 为后续的数据清洗制定策略

```python
# 查看数据形状（行数，列数）
print(df_order.shape)

# 查看列名
print(df_order.columns)

# 查看数据类型
print(df_order.dtypes)
```

Listing 4: 查看数据形状和列名

shape 属性的含义

df.shape 返回一个元组（行数，列数），表示数据的维度。

示例：

```
>>> df.shape
(50, 9)  # 表示50行，9列
```

4.2 查看数据内容

```python
# 查看前5行
print(df_order.head())

# 查看后5行
print(df_order.tail())

# 查看前10行
print(df_order.head(10))
```

Listing 5: 查看数据内容

head() 和 tail() 的用法

- `head(n)`：查看前 n 行，默认 n=5
- `tail(n)`：查看后 n 行，默认 n=5

查看数据的开头可以了解数据结构，查看结尾可以检查数据是否完整导入。

4.3 统计信息

描述性统计

描述性统计是对数据特征进行总结和描述的方法。通过统计量，我们可以快速了解数据的分布情况。

常用统计量：

- **count**：非空值数量
- **mean**：平均值
- **std**：标准差（数据离散程度）
- **min**：最小值
- **25%**：第一四分位数（Q1）
- **50%**：中位数
- **75%**：第三四分位数（Q3）
- **max**：最大值

```python
# 查看数值列的统计信息
print(df_order.describe())

# 查看缺失值统计
print(df_order.isnull().sum())
```

Listing 6: 使用 describe() 查看统计信息

知识点：isnull() 和 sum()

`isnull()` 函数用于检测缺失值，返回一个布尔值的 DataFrame，缺失值位置为 True。

`sum()` 函数对布尔值进行求和时，True 被当作 1，False 被当作 0，因此可以直接统计每列的缺失值数量。

执行流程：

1. `df.isnull()` → 生成布尔值表格
2. `.sum()` → 对每列求和（统计 True 的数量）

动手练习 2：探索传感器数据

任务： 使用 02_ 传感器数据 _ 练习.csv，查看数据的基本统计信息。

```python
import pandas as pd

df_sensor = pd.read_csv('02_传感器数据_练习.csv')

# 查看形状
print("数据形状：", df_sensor.shape)

# 查看统计信息
print("\n统计摘要：")
print(df_sensor.describe())

# 查看有多少个不同的传感器
print("\n传感器数量：", df_sensor['sensor_id'].nunique())
print("传感器列表：", df_sensor['sensor_id'].unique())
```

5 数据选择

5.1 选择列

DataFrame 的列选择

在 Pandas 中，选择列是最常用的操作之一。DataFrame 可以看作是由多个 Series（列）组成的字典。

两种选择方式：

1. **单列选择**：返回一个 Series 对象
2. **多列选择**：返回一个 DataFrame 对象

```
# 选择单列（商品名称）
products = df_order['product_name']

# 选择多列（订单ID、商品名称、价格）
order_info = df_order[['order_id', 'product_name', 'price']]
```

Listing 7: 选择单列和多列

注意方括号的区别

- `df[' 列名']` ——单层方括号，选择单列，返回 Series
- `df[[' 列 1', ' 列 2']]` ——双层方括号，选择多列，返回 DataFrame

5.2 选择行

iloc 和 loc 的区别

Pandas 提供了两种主要的行选择方式：

1. **iloc**：基于**整数位置**索引（从 0 开始）
2. **loc**：基于**标签**索引（使用行标签）

对于刚读取的 CSV 文件，行标签默认就是整数 0, 1, 2...，所以 iloc 和 loc 看起来效果相似。但当数据经过筛选、排序后，行标签可能不再是连续的整数，这时两者的区别就会显现。

```
# 选择第1行（索引为0）
```

```
row = df_order.iloc[0]

# 选择前5行
rows = df_order.iloc[0:5]

# 选择第10到20行
rows = df_order.iloc[10:20]
```

Listing 8: 使用 iloc 选择行

5.3 条件筛选

布尔索引

条件筛选是数据分析中最强大的功能之一。它的原理是**布尔索引**:

1. 首先创建一个布尔条件（如 `df['age'] > 18`）
2. 这个条件会生成一个布尔值的 Series
3. 将这个布尔 Series 放入方括号中，Pandas 会只返回 True 对应的行

常用比较运算符:

- `==` 等于
- `!=` 不等于
- `>` 大于
- `<` 小于
- `>=` 大于等于
- `<=` 小于等于

```
# 筛选数量大于1的订单
large_orders = df_order[df_order['quantity'] > 1]

# 筛选特定商品
iphone_orders = df_order[df_order['product_name'] == 'iPhone 14
    Pro']
```

Listing 9: 条件筛选示例

动手练习 3：筛选缺失值数据

任务： 使用 03_ 缺失值处理 _ 练习.csv，筛选出年龄缺失的员工。

```python
import pandas as pd

df_missing = pd.read_csv('03_缺失值处理_练习.csv')

# 筛选年龄缺失的行
missing_age = df_missing[df_missing['age'].isnull()]

print("年龄缺失的员工：")
print(missing_age[['name', 'age', 'department']])

print(f"\n共 {len(missing_age)} 人年龄缺失")
```

6 数据清洗：处理缺失值

6.1 什么是缺失值？

缺失值的产生原因

缺失值（Missing Values）是指数据集中某些记录在某些字段上没有值。缺失值在真实数据中非常常见。

缺失值的常见原因：

1. **数据采集失败**：设备故障、网络中断
2. **数据录入遗漏**：人工录入时漏填
3. **信息不适用**：某些字段对某些记录不适用
4. **隐私保护**：敏感信息被故意隐藏
5. **数据合并问题**：多个数据源合并时字段不匹配

缺失值的表示方式：

- Python 中：`None` 或 `np.nan`（Not a Number）

- CSV 文件中：空单元格或特定标记（如"N/A"、"NULL"）
- 数据库中：NULL

6.2 检测缺失值

```python
# 统计每列的缺失值数量
print(df_missing.isnull().sum())

# 计算缺失值比例
missing_ratio = df_missing.isnull().sum() / len(df_missing) * 100
print(missing_ratio)
```

Listing 10: 检测缺失值

缺失值检测方法

- df.isnull()：返回布尔值 DataFrame，缺失值为 True
- df.notnull()：返回布尔值 DataFrame，非缺失值为 True
- df.isnull().sum()：统计每列缺失值数量
- df.isnull().sum().sum()：统计整个 DataFrame 的缺失值总数

6.3 缺失值处理策略

缺失值处理策略

处理缺失值没有绝对正确的方法，需要根据具体情况选择策略：

策略一：删除缺失值

- 适用场景：缺失值比例很小（如 <5%），且缺失是随机的
- 优点：简单直接
- 缺点：可能丢失有价值的信息

策略二：填充缺失值

- 适用场景：缺失值比例较大，或数据很珍贵不能删除
- 填充方法：

- 固定值填充（如 0、”未知”）
- 统计值填充（均值、中位数、众数）
- 前后值填充（时间序列数据）
- 模型预测填充（高级方法）

策略三：标记缺失

- 创建一个新列标记哪些值是填充的
- 适用于缺失本身可能有意义的情况

6.4 填充缺失值

```python
# 用固定值填充
df_missing['gender'] = df_missing['gender'].fillna('未知')

# 用平均值填充（工资列）
mean_salary = df_missing['salary'].mean()
df_missing['salary'] = df_missing['salary'].fillna(mean_salary)

# 用中位数填充（年龄列）
median_age = df_missing['age'].median()
df_missing['age'] = df_missing['age'].fillna(median_age)

# 用众数填充（部门列）
mode_dept = df_missing['department'].mode()[0]
df_missing['department'] = df_missing['department'].fillna(
    mode_dept)
```

Listing 11: 填充缺失值的方法

均值、中位数、众数的选择

- **均值（Mean）**：适用于数据分布对称的情况，受异常值影响大
- **中位数（Median）**：适用于有异常值或分布偏斜的情况，更稳健
- **众数（Mode）**：适用于分类数据，表示出现频率最高的值

动手练习 4：处理缺失值

任务： 使用 03_ 缺失值处理 _ 练习.csv，完成缺失值处理。

```python
import pandas as pd

df = pd.read_csv('03_缺失值处理_练习.csv')

print("处理前缺失值统计：")
print(df.isnull().sum())

# 用中位数填充年龄
median_age = df['age'].median()
df['age'] = df['age'].fillna(median_age)

# 用众数填充性别
mode_gender = df['gender'].mode()[0]
df['gender'] = df['gender'].fillna(mode_gender)

# 用平均值填充工资
mean_salary = df['salary'].mean()
df['salary'] = df['salary'].fillna(mean_salary)

print("\n处理后缺失值统计：")
print(df.isnull().sum())
```

7 数据清洗：处理重复值

7.1 什么是重复值？

重复值的产生原因

重复值是指数据集中完全相同的记录出现了多次。

重复值的常见原因：

1. **数据录入错误**：同一条数据被重复录入
2. **数据合并**：多个数据源合并时产生重复
3. **系统问题**：网络重传、事务重复提交

4. **定时任务**：数据同步任务重复执行

重复值的影响：

- 导致统计分析结果偏差
- 增加存储和计算成本
- 影响机器学习模型训练

7.2 检测和删除重复值

```python
# 检测重复值
duplicates = df_order.duplicated()
print(f"重复行数：{duplicates.sum()}")

# 查看重复的行
duplicate_rows = df_order[df_order.duplicated(keep=False)]
print(duplicate_rows)

# 删除重复值（保留第一个）
df_order_clean = df_order.drop_duplicates()
```

Listing 12: 处理重复值

duplicated() 和 drop_duplicates() 详解

`duplicated()` 函数：

- 返回布尔值 Series，标记每行是否重复
- `keep='first'`：第一次出现标记为 False，后续重复标记为 True
- `keep='last'`：最后一次出现标记为 False，前面重复标记为 True
- `keep=False`：所有重复行都标记为 True

`drop_duplicates()` 函数：

- 删除重复行，返回新的 DataFrame
- `keep='first'`：保留第一次出现的行（默认）
- `keep='last'`：保留最后一次出现的行

- keep=False：删除所有重复行

动手练习 5：处理电商订单重复值

任务：使用 01_ 电商订单数据 _ 练习.csv，检测并删除重复订单。

```python
import pandas as pd

df = pd.read_csv('01_电商订单数据_练习.csv')

print(f"原始数据行数：{len(df)}")

# 检测重复值
n_duplicates = df.duplicated().sum()
print(f"重复行数：{n_duplicates}")

# 查看重复的行
duplicate_rows = df[df.duplicated(keep=False)]
print("\n重复的行：")
print(duplicate_rows[['order_id', 'product_name', 'price']])

# 删除重复值
df_clean = df.drop_duplicates()
print(f"\n删除后行数：{len(df_clean)}")
```

8 数据清洗：处理异常值

8.1 什么是异常值？

异常值的定义和类型

异常值（Outliers）是指与数据集中大多数观测值明显不同的数据点。

异常值的类型：

1. **错误值**：数据采集或录入错误（如年龄为负数）
2. **真实异常**：真实存在但罕见的情况（如收入极高的人）
3. **特殊情况**：特定条件下的正常值（如促销期间的销量激增）

异常值的影响：

- 使均值、标准差等统计量失真
- 影响机器学习模型的训练效果
- 可能导致错误的业务决策

8.2 异常值检测方法

IQR 方法（箱线图法）

IQR（Interquartile Range，四分位距）是一种常用的异常值检测方法。

计算步骤：

1. 计算第一四分位数 Q1（25% 分位数）
2. 计算第三四分位数 Q3（75% 分位数）
3. 计算 IQR = Q3 - Q1
4. 定义异常值范围：
   - 下限 = Q1 - 1.5 × IQR
   - 上限 = Q3 + 1.5 × IQR
5. 超出此范围的值被认为是异常值

为什么是 1.5 倍？这是统计学中的经验法则，约涵盖 99.3% 的正态分布数据。

```python
# 计算IQR
Q1 = df_order['quantity'].quantile(0.25)
Q3 = df_order['quantity'].quantile(0.75)
IQR = Q3 - Q1

# 定义异常值范围
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# 找出异常值
outliers = df_order[(df_order['quantity'] < lower_bound) |
                    (df_order['quantity'] > upper_bound)]
```

```python
print(f"异常值数量: {len(outliers)}")
```

Listing 13: 使用 IQR 方法检测异常值

8.3 处理异常值

```python
# 将负数数量修正为1
df_order.loc[df_order['quantity'] < 0, 'quantity'] = 1
```

Listing 14: 处理异常值

异常值处理策略

1. **删除**：如果确定是错误值，直接删除
2. **修正**：根据业务逻辑修正（如负数数量改为正数）
3. **替换**：用边界值或统计值替换
4. **保留**：如果是真实异常情况，保留并单独分析
5. **分箱**：将异常值归入特殊类别

9 数据格式转换

9.1 为什么需要格式转换？

数据类型的重要性

正确的数据类型对于数据分析至关重要：

- **数值计算**：价格必须是数值型才能进行加减乘除
- **内存效率**：正确的类型可以节省内存
- **函数兼容性**：某些函数只接受特定类型
- **排序和比较**：字符串"10" 小于"2"，但数字 10 大于 2

常见类型问题：

1. 数字被存储为字符串（如"123 元"）
2. 日期被存储为字符串

3. 分类数据没有统一格式

9.2 字符串处理

```
# 查看价格列的格式
print(df_order['price'].head())

# 去除"元"字，转换为数值
df_order['price_num'] = df_order['price'].str.replace('元', '')
df_order['price_num'] = df_order['price_num'].astype(float)
```

Listing 15: 处理价格列

字符串方法 str.XXX

Pandas 的 Series 对象有一个 `str` 属性，提供了一系列字符串处理方法：

- `str.replace(a, b)`：将 a 替换为 b
- `str.strip()`：去除首尾空格
- `str.lower()` / `str.upper()`：大小写转换
- `str.contains()`：判断是否包含某子串
- `str.split()`：分割字符串

9.3 处理手机号

```
# 去除横线
df_order['phone_clean'] = df_order['phone'].str.replace('-', '')
```

Listing 16: 标准化手机号

动手练习 6：清洗格式混乱数据

任务： 使用 04_ 数据格式清洗 _ 练习.csv，清洗姓名和收入列。

```
import pandas as pd

```

```python
df = pd.read_csv('04_数据格式清洗_练习.csv')

# 查看原始数据
print("姓名处理前：")
print(df['name'].head())

# 去除姓名中的空格
df['name_clean'] = df['name'].str.strip()

print("\n姓名处理后：")
print(df['name_clean'].head())

# 处理收入（将12k转换为12000）
def convert_income(income_str):
    income_str = str(income_str).strip()
    if 'k' in income_str.lower():
        number = float(income_str.lower().replace('k', ''))
        return number * 1000
    else:
        return float(income_str)

df['income_num'] = df['income'].apply(convert_income)

print("\n收入转换：")
print(df[['income', 'income_num']].head(10))
```

10 数据分组与统计

10.1 分组统计的概念

什么是分组统计？

分组统计（GroupBy）是数据分析中最强大的功能之一，它的思想是：

”先分组，再统计”

类比理解：

- 就像老师统计每个班级的平均分
- 先按班级**分组**，再计算每组的**平均分**

SQL 中的等价概念：

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

10.2 分组统计

```python
# 按部门分组，计算平均工资
dept_avg = df_missing.groupby('department')['salary'].mean()
print(dept_avg)

# 多列分组统计
stats = df_missing.groupby('department').agg({
    'salary': ['mean', 'max', 'min'],
    'age': 'mean'
})
print(stats)
```

Listing 17: 按部门统计工资

groupby() 详解

groupby() 的基本流程：

1. **Split**：按指定列的值将数据分组
2. **Apply**：对每个组应用统计函数
3. **Combine**：将结果合并成新的 DataFrame

常用聚合函数：

- mean()：平均值
- sum()：求和
- count()：计数
- max() / min()：最大/最小值
- std()：标准差

动手练习 7：统计电商数据

任务： 使用 01_ 电商订单数据 _ 练习.csv，统计各商品的总销量。

```python
import pandas as pd

df = pd.read_csv('01_电商订单数据_练习.csv')

# 处理价格（去除"元"字）
df['price_num'] = df['price'].str.replace('元', '').astype(
    float)

# 计算订单金额
df['total_amount'] = df['price_num'] * df['quantity']

# 按商品分组统计
product_stats = df.groupby('product_name').agg({
    'quantity': 'sum',
    'total_amount': 'sum'
}).sort_values('total_amount', ascending=False)

print("商品销售统计TOP10：")
print(product_stats.head(10))
```

11 综合实战：完整数据处理流程

11.1 数据处理的完整流程

标准数据处理流程

一个完整的数据处理项目通常遵循以下流程：

1. **数据读取**：从文件或数据库加载数据
2. **数据探索**：了解数据的基本情况
3. **数据清洗**：
   - 处理缺失值
   - 处理重复值

- 处理异常值
- 格式转换

4. **特征工程**：创建新特征、转换现有特征
5. **数据分析**：统计分析、可视化
6. **结果保存**：保存清洗后的数据

11.2 案例：完整的电商订单数据处理

```python
import pandas as pd
import numpy as np

# 第1步：读取数据
df = pd.read_csv('01_电商订单数据_练习.csv')
print(f"原始数据：{len(df)}行")

# 第2步：删除重复值
df = df.drop_duplicates()
print(f"删除重复后：{len(df)}行")

# 第3步：处理缺失值
df['user_id'] = df['user_id'].fillna('UNKNOWN')

# 第4步：处理异常值（负数数量）
df.loc[df['quantity'] < 0, 'quantity'] = 1

# 第5步：格式转换（价格）
df['price_num'] = df['price'].str.replace('元', '').astype(float
    )

# 第6步：格式转换（手机号）
df['phone_clean'] = df['phone'].str.replace('-', '')

# 第7步：计算订单金额
df['total_amount'] = df['price_num'] * df['quantity']

# 第8步：统计分析
```

```python
product_sales = df.groupby('product_name')['total_amount'].sum()
print("\n各商品销售额：")
print(product_sales.sort_values(ascending=False).head())

# 第9步：保存结果
df.to_csv('清洗后的订单数据.csv', index=False)
print("\n数据已保存到：清洗后的订单数据.csv")
```

Listing 18: 完整的数据处理流程

12 云端仿真平台实战

这一章要做什么？

前面的练习用的是本地 CSV 文件。这一章教你从云端数据仿真平台实时获取数据，用 Pandas 做分析。

仿真平台地址：`http://163.7.1.176:8889`

平台每秒自动生成电商用户行为数据（浏览、点击、加购、购买等），你可以通过 API 获取。

12.1 用 requests 获取数据

```python
import requests
import pandas as pd

# 从仿真平台获取 500 条数据
response = requests.get("http://163.7.1.176:8889/api/data?n=500")
data = response.json()  # 解析 JSON

# 转换为 DataFrame
df = pd.DataFrame(data)

# 查看数据
print(df.shape)          # (500, 11)
print(df.columns)        # 列名
print(df.head())         # 前5行
print(df.dtypes)         # 数据类型
```

12.2 数据清洗练习

练习 1：从仿真平台获取数据并清洗

```python
import requests
import pandas as pd

# 获取 1000 条数据
df = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/data?n=1000").json())

# 练习1：检查缺失值
print(df.isnull().sum())

# 练习2：price 列有缺失值（不是所有事件都有价格）
# 用 0 填充缺失的价格
df['price'] = df['price'].fillna(0)

# 练习3：将 timestamp 转为日期类型
df['timestamp'] = pd.to_datetime(df['timestamp'])

# 练习4：提取小时
df['hour'] = df['timestamp'].dt.hour

# 练习5：按事件类型统计
print(df['event_type'].value_counts())
```

12.3 数据分析练习

练习 2：分析仿真平台数据

```python
import requests
import pandas as pd

df = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/data?n=2000").json())

# 练习1：各品类的平均价格
print(df.groupby('category')['price'].mean().sort_values(
```

```python
    ascending=False))

# 练习2：各设备的使用占比
print(df['device'].value_counts(normalize=True))

# 练习3：购买记录筛选
buys = df[df['event_type'] == 'buy']
print(f"购买记录：{len(buys)} 条")
print(buys.groupby('category')['price'].sum().sort_values(
    ascending=False))

# 练习4：各来源页面的转化情况
pivot = df.pivot_table(values='user_id', index='page_source
    ',
                       columns='event_type', aggfunc='count
                           ', fill_value=0)
print(pivot)
```

12.4 导出清洗后的数据

```python
# 清洗后导出为 CSV（后续大数据分析会用到）
df_clean = df.dropna(subset=['price'])
df_clean.to_csv('cleaned_behavior.csv', index=False, encoding='
    utf-8-sig')
print(f"已导出 {len(df_clean)} 条清洗后的数据")
```

衔接下一步

清洗后的数据 `cleaned_behavior.csv` 可以上传到大数据平台（HDFS），在《大数据分析实战手册》中用 Spark SQL 进一步分析。

13 竞赛模拟题

13.1 题目一：电商订单数据清洗

竞赛模拟题 1

题目要求：

使用 **01_ 电商订单数据 _ 练习.csv**，完成以下任务：

1. 读取数据，查看基本信息（形状、列名、前 5 行）
2. 统计缺失值数量
3. 删除重复订单
4. 处理 user_id 缺失值（用'UNKNOWN' 填充）
5. 处理负数数量（修正为 1）
6. 提取价格中的数字，转换为数值型
7. 标准化手机号格式（去除横线）
8. 计算每个订单的总金额（价格 × 数量）
9. 统计每个商品的总销售额和总销量
10. 保存清洗后的数据到新的 CSV 文件

参考答案：

```python
import pandas as pd

# 读取数据
df = pd.read_csv('01_电商订单数据_练习.csv')

# 查看基本信息
print(f"数据形状：{df.shape}")
print(f"列名：{df.columns.tolist()}")
print(df.head())

# 统计缺失值
print("\n缺失值统计：")
print(df.isnull().sum())
```

```python

# 删除重复值
df = df.drop_duplicates()

# 填充缺失值
df['user_id'] = df['user_id'].fillna('UNKNOWN')

# 处理异常值
df.loc[df['quantity'] < 0, 'quantity'] = 1

# 格式转换
df['price_num'] = df['price'].str.replace('元', '').astype(
    float)
df['phone_clean'] = df['phone'].str.replace('-', '')

# 计算金额
df['total_amount'] = df['price_num'] * df['quantity']

# 统计分析
product_stats = df.groupby('product_name').agg({
    'total_amount': 'sum',
    'quantity': 'sum'
}).sort_values('total_amount', ascending=False)

print("\n商品销售统计: ")
print(product_stats.head(10))

# 保存结果
df.to_csv('清洗后的订单数据.csv', index=False)
```

13.2 题目二：员工信息数据处理

竞赛模拟题 2

题目要求：

使用 **03_ 缺失值处理 _ 练习.csv**，完成以下任务：

1. 读取数据，查看缺失值情况

2. 用中位数填充 age 列的缺失值
3. 用平均值填充 salary 列的缺失值
4. 用众数填充 department 列的缺失值
5. 用众数填充 gender 列的缺失值
6. 删除 score 列仍有缺失值的行
7. 按 department 分组，统计平均工资和平均年龄
8. 保存处理后的数据

参考答案:

```python
import pandas as pd

# 读取数据
df = pd.read_csv('03_缺失值处理_练习.csv')

# 查看缺失值
print("缺失值统计: ")
print(df.isnull().sum())

# 用中位数填充年龄
median_age = df['age'].median()
df['age'] = df['age'].fillna(median_age)

# 用平均值填充工资
mean_salary = df['salary'].mean()
df['salary'] = df['salary'].fillna(mean_salary)

# 用众数填充部门和性别
mode_dept = df['department'].mode()[0]
df['department'] = df['department'].fillna(mode_dept)

mode_gender = df['gender'].mode()[0]
df['gender'] = df['gender'].fillna(mode_gender)

# 删除score缺失的行
```

```python
df = df.dropna(subset=['score'])

# 按部门统计
dept_stats = df.groupby('department').agg({
    'salary': 'mean',
    'age': 'mean'
}).round(2)

print("\n部门统计: ")
print(dept_stats)

# 保存结果
df.to_csv('处理后的员工数据.csv', index=False)
```

14 代码速查表

14.1 文件操作

```python
# 读取CSV
df = pd.read_csv('文件名.csv')

# 保存CSV
df.to_csv('新文件名.csv', index=False)
```

Listing 19: 读取和保存 CSV

14.2 数据查看

```python
df.shape            # 形状 (行数, 列数)
df.columns          # 列名
df.head()           # 前5行
df.tail()           # 后5行
df.info()           # 详细信息
df.describe()       # 统计摘要
df.dtypes           # 数据类型
```

Listing 20: 数据查看

14.3 数据选择

```
# 选择列
df['列名']
df[['列1', '列2']]

# 选择行
df.iloc[0]          # 第1行
df.iloc[0:5]        # 前5行

# 条件筛选
df[df['列名'] > 值]
```

Listing 21: 数据选择

14.4 数据清洗

```
# 缺失值
df.isnull().sum()                    # 统计缺失值
df['列'].fillna(值)                  # 填充缺失值
df.dropna()                          # 删除缺失值

# 重复值
df.duplicated().sum()                # 统计重复值
df.drop_duplicates()                 # 删除重复值

# 异常值 (IQR方法)
Q1 = df['列'].quantile(0.25)
Q3 = df['列'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['列'] < Q1-1.5*IQR) | (df['列'] > Q3+1.5*IQR)]
```

Listing 22: 数据清洗

14.5 数据转换

```
# 类型转换
df['列'].astype(float)
```

```python
df['列'].astype(str)

# 字符串处理
df['列'].str.replace('旧', '新')
df['列'].str.strip()

# 应用函数
df['新列'] = df['列'].apply(函数)
```

Listing 23: 数据转换

14.6 分组统计

```python
# 简单分组
df.groupby('分组列')['统计列'].mean()

# 多统计量
df.groupby('分组列').agg({
    '列1': 'mean',
    '列2': ['mean', 'sum']
})
```

Listing 24: 分组统计